

Quickstart — Claude Code for Research

Welcome. This is the two-minute version: just enough to be productive in your very first session. When you want the full story behind any point below, the Detailed Guide (`USAGE_DETAILED.md`) expands all of it.

What It Is

Claude Code is an agentic pair-scientist that lives in your terminal. It reads and writes files and runs code directly in your repository, working in turns: you set a goal, it plans and acts, and you review the result. It's safe by default — the toolkit blocks destructive commands, so nothing catastrophic happens behind your back. (See “The Map — What Claude Code Is, and Its Architecture” in the Detailed Guide.)

The Day-One List

Fifteen things to know on your first day:

1. Launch. `cd` into your project and run `claude`. That repository's `CLAUDE.md` and rules load automatically. (See “The Loop, and Driving It.”)
2. Prompt. Type an instruction and press Enter. Esc interrupts, and you can steer at any time by typing a correction mid-task. Be specific, and name your output paths. (See “The Loop, and Driving It.”)
3. Modes — the one keystroke that matters. Shift+Tab cycles through normal, auto-accept edits, and plan mode. Use plan for anything nontrivial, and trust the label above the box to tell you which mode you're in. (See “The Loop, and Driving It” and “Actions and Guardrails.”)
4. Models. `claude-fable-5[1m]` is preset; use `/model` to switch. Fable 5 and Opus 4.8 are for hard problems, Sonnet for routine work, Haiku for trivial edits. Never Claude Opus 5, and never the bare opus alias (it resolves to Opus 5) — use full IDs. (See “Instructions — How You Steer It.”)
5. Effort. Preset to MAX — nothing for you to do. (See “Instructions — How You Steer It.”)
6. Permissions. You'll get prompts. The toolkit blocks `rm`, `curl`, `sudo`, and the like for safety — allow the safe ones, or add them to `.claude/settings.local.json`. (See “Actions and Guardrails.”)
7. Slash commands. `/help`, `/clear` (new topic), `/context`, `/model`, `/baton` (handoff), `/folio` (make a PDF; add “docx” for a Word file), `/solo` (run autonomously, no check-ins), and `/xbeep` (beeps). Type `/` for the full menu. These commands also take trailing text as instructions — e.g. `/compact focus: keep the file paths we're editing`. (See “Instructions — How You Steer It.”)
8. Skills auto-fire. Just describe the task — fit a GAM, gap-fill, QC a series, build a brms model — and the right skill loads itself. Force one with `/name` if you like. (See “Instructions — How You Steer It.”)
9. Interactive by default. Give a clear goal and Claude moves briskly — the always-on autonomy and no-check-in rules keep it from asking “should I continue?” after every trivial step, so it won't nag — but on a normal session, expect it to check in at natural decision points. When you want it fully hands-off, running all the way through with no stops, hand the task off with `/solo`. (See “Part B — The Toolkit Overlay” and “Instructions — How You Steer It.”)
10. Sandbox. All test and scratch files go into a `sandbox/` directory, keeping your repo clean. (See “Part B — The Toolkit Overlay.”)
11. Always-on rules do the worrying for you. Claude verifies before saying “done,” fixes root causes rather than symptoms, and checks facts from their source. You can rely on all of this as given. (See “Part B — The Toolkit Overlay.”)
12. Beeps mean one of three things: a prompt was sent, Claude finished, or it needs a permission. Toggle them with `/xbeep`. (See “Automation.”)
13. Context auto-summarizes as it fills. For a clean break, write a handoff with `/baton`, then `/clear` between unrelated tasks. (See “Context — What Claude Sees.”)
14. Long runs go to the background — you get a beep or push notification when they finish, and Claude keeps working meanwhile. (See “Delegation and Scale.”)
15. Restart to apply setting or hook changes — they load at startup. (See “Troubleshooting and FAQ.”)

Your First Session (the recipe)

cd into your repo, run `c laude`, and press Shift+Tab to reach plan mode. Describe the task — data path, model, output path — then read and approve the plan Claude returns. Let it work, review the results, run `/baton` to save a handoff, and `/clear` before the next task. (See “Your First Research Session, Step by Step.”)

For More

Everything above expands in the Detailed Guide (`USAGE_DETAILED.md`) — follow the section pointers in parentheses. Start there for the internals of permissions, the full set of twenty-one skills and five agents, the effort levels, the context tools, the hooks, the research patterns, and the glossary.